

xlab内训 宝盖版

about github-desktop:

- clone: file-clone
- 保存修改: commit to main(到master) + fetch origin
 - commit to main处的summary会变成github网页上的注释。还是要好好写以清晰明了的表达。
- 在vscode编辑时, `.c` 文件旁边的 **M** 代表 **Modified** (已修改)。这表示该文件自上次 Git 提交(commit) 后已经被更改, 但还没有被提交到本地仓库。

about 项目链

- 打造一个规范的工程模板
 - 一个规范的工程模板应该包含以下几个文件夹/文件:
 - inc 文件夹: 头文件 (.h 文件)
 - src 文件夹: 源文件 (.c 文件)
 - lib 文件夹: 库文件
 - bin 文件夹: 可执行文件
 - build 文件夹: 编译过程中生成的文件
 - docs 文件夹: 文档
 - CMakeLists.txt: CMake 的配置文件/Makefile: Make 的配置文件
 - README.md: 项目说明
- `.h` 文件规范(xx.h)

```
1  #ifndef _XX_H_
2  #define _XX_H_
3  ...
4  #endif
```
- about Makefile:

Makefile 是一种构建工具，它可以自动化执行编译、链接等操作，使得代码的编译和运行更加方便。

Makefile 的基本结构如下：

- 目标：依赖
- 命令

例如：

```
1 main: main.o func.o
2     gcc -o main main.o func.o
3 main.o: main.c
4     gcc -c main.c
5 func.o: func.c
6     gcc -c func.c
```

- about CMakeLists.txt:

CMake 是一个跨平台的构建工具，它可以自动生成 Makefile、Visual Studio 项目、Xcode 项目等，使得代码的编译和运行更加方便。

CMake 的基本结构如下：

```
1 PROJECT (MAIN)
2 SET(SRC_LIST main.c print1.c print.c)
3 add_executable(main ${SRC_LIST})
```

CMake 的编译流程

- 进入到 build 目录下：cd build
- 运行 cmake 的命令：cmake .. (.. 是指上一级目录，也就是说，我们要在 build 目录下运行它上一级目录也就是我们的工程文件夹下的 cmake，这样做的目的是把 cmake 生成的乱七八糟的文件都放在 build 目录下)
- 这时已经可以看到 build 目录下生成了 Makefile 文件，之后我们去运行这个 Makefile
- 最后我们即可发现在 bin 目录下生成了我们需要的可执行文件

不要让你的路径出现中文，不然你会像我一样调一晚上也不知道哪出了问题

推荐插件：CMake

写完cmakelists.txt, 在终端输入 `cmake .` (或 `cmake --build .?`) 有空格!! 能通过vscode左下角的“生成”键来生成最好。

about ESP32

- 当做正常的单片机就好啦。
- vscode上先 build (√) 再 upload (→) 。
- 一个小点：upload 若显示 A fatal error occurred: Could not open COM10, the port doesn't exist, 可能串口被占用，这时候检查是不是打开了串口调试助手没有关。要看隐藏的图标有没有彻底关。

about EDA与电路

1. 流程

- 仿真 (?)
- 原理图
- pcb走线
 - 电源线粗，信号线窄。
 - 底层，顶层走线。贴片电阻。
 - 不要直角。
 - 先（隐藏gnd飞线）连线；泪滴；（gnd打过孔）铺铜（gnd）；检查drc；

2. 电路知识

- cmos管： [MOS管基础-CSDN博客](#)

about 机电部分

- SolidWorks打不开：
先remove 后install。管理员身份运行。

